CHAPTER 06

Design Consistent Hashing

Distribute keys across a changing set of servers while moving as few keys as possible.

Jump to: Problem statement | Requirements | Back-of-the-envelope estimation | High-level design | Deep dive | Trade-offs and alternatives | Scaling, reliability, failure modes | Likely follow-up questions | Common mistakes | Interview answer script | 60-second recap / practice checklist | References and further reading

1. Problem statement

To scale horizontally, requests/data must be distributed evenly across N servers (cache nodes, shards). The naive approach `server = hash(key) % N` breaks when servers are added or removed: almost **all keys remap**, causing a storm of cache misses. Design a hashing scheme that keeps the distribution even and **only remaps about k/N keys** when a node joins or leaves.

Why interviewers ask it: Foundational building block used by caches, KV stores, CDNs, load balancers (Dynamo, Cassandra, Memcached clients).

2. Requirements

Functional

- Map a key to a server deterministically.
- Add or remove servers dynamically.
- Spread keys (and load) evenly across servers.

Non-functional

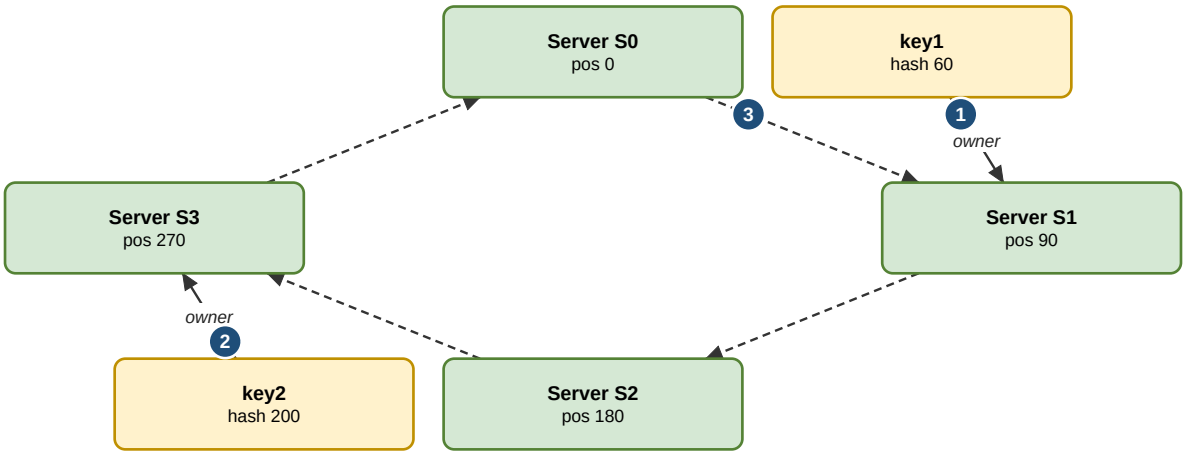
- **Minimal data movement** on membership change (k/N keys).
- **Balance:** low variance in keys per server.
- **Fast lookup:** O(log N) or better.
- Resilient to hot nodes and heterogeneous server capacity.

3. Back-of-the-envelope estimation

- With 4 servers and `hash % 4`, removing one server changes the divisor to 3: about 75% of keys now map to a different server.
- With consistent hashing, removing 1 of N servers moves only about 1/N of keys (those the server owned).
- Ring lookup: sorted array/tree of M virtual node positions (M = N x vnodes e.g. 100 x 200 = 20 K) -> binary search about 15 steps.

4. High-level design

Hash ring: keys walk clockwise to the next server



The ring is the hash space 0..2^32-1 joined end to end. Servers and keys are hashed onto it; a key belongs to the first server clockwise.

Step	What happens
1	Hash `key1` onto the ring (position 60). Walk **clockwise** from there until you meet the first server.
2	Hash `key2` (position 200). The next server clockwise is S3 (position 270), so S3 owns it.
3	**Add or remove a server:** only the arc between the new server and its predecessor moves. Removing S1 would hand its keys to S2, and nothing else changes.

5. Deep dive

Why modulo hashing fails

- `serverIndex = hash(key) % N` is perfect while N is constant.
- When a server dies (N changes) most keys map elsewhere -> cache stampede on the database, or massive data migration in a sharded store.

The hash ring

- Use a hash function (SHA-1, MurmurHash) with a huge output space, e.g. 0 .. 2^160-1 (or 2^32-1), and bend it into a circle.
- Hash **servers** (by IP or name) onto the ring. Hash **keys** onto the same ring.
- Lookup: go clockwise from the key's position until hitting a server; that server owns the key.
- Add server: it takes over keys between its predecessor and itself - only that range moves. Remove server: its range moves to the next server clockwise.

Two problems with the basic ring

- **Uneven partitions:** it is impossible to keep partition sizes equal when servers join/leave; one server may own a huge arc.
- **Non-uniform key distribution:** keys may cluster on one arc, overloading a server.

Virtual nodes (replicas) - the fix

- Each physical server is hashed onto the ring many times (`S0#1`, `S0#2`, ..., e.g. 100-200 vnodes). A key belongs to the physical server of the first vnode clockwise.
- More vnodes -> lower standard deviation of load (about 5-10% with 100-200 vnodes), at the cost of more metadata memory. Tunable.
- Heterogeneous machines: give bigger machines more vnodes.
- On failure, the dead server's many small arcs are spread across **many** other servers rather than dumping everything on one neighbour.

Finding affected keys

- **Add node:** walk anticlockwise from the new node to the previous node; keys in that range move to the new node.
- **Remove node:** keys in the range between its predecessor and itself move to its successor.

```
ring = sorted list of (hash(server#i), server)
lookup(key): h = hash(key); idx = bisect_right(ring_hashes, h) % len(ring); return ring[idx].server
```

Replication on the ring (Dynamo style)

- For N-way replication, store the key on the next N **distinct physical** servers clockwise (skip vnodes of the same server).
- Combined with quorum reads/writes (see key-value store chapter).

Real-world usage

- Amazon Dynamo and Apache Cassandra partitioning; Discord/Akamai CDN request routing; Maglev and other load balancers; Memcached client-side sharding.
- Variants: rendezvous (highest random weight) hashing; **jump consistent hash** (no memory, needs sequential bucket numbers); bounded-load consistent hashing.

6. Trade-offs and alternatives

Decision	Option A / Option B	How to choose
Mod-N vs consistent hash	Simple, perfectly balanced, O(1) / Minimal movement, needs ring structure	Mod-N only if cluster size never changes; else consistent hashing.
Number of vnodes	Few: low memory, imbalance / Many: even load, larger ring + slower rebalancing	100-256 per node is a common balance.
Ring vs rendezvous vs jump hash	Ring: general, vnodes / Rendezvous: simple, O(N) per lookup / Jump: no state, append-only buckets	Ring for dynamic nodes; jump hash for sharded stores with numbered shards.

7. Scaling, reliability, failure modes

- Keep the ring metadata (membership) in a coordination service (ZooKeeper/etcd) or gossip; clients cache it.
- Rebalancing: move data in the background; serve reads from both old and new owner until done.
- Hot keys are not solved by hashing: use replication of hot keys, local caches, or key salting.

8. Likely follow-up questions

- How many keys move when a node is added? (about k/N)
- How do vnodes help heterogeneous hardware and failure recovery?
- How do you handle a hot key / celebrity user?
- How do clients learn about ring changes? (gossip, config service)

9. Common mistakes

- Using too few vnodes and claiming 'perfectly balanced'.
- Choosing a poor or non-uniform hash function.
- Placing replicas on vnodes of the same physical machine.

10. Interview answer script (what to say, in order)

Follow the 4-step framework from chapter 04. Say it out loud; the quoted text is a model answer in your own voice.

Phase	Time	What to say
1. Understand + scope	3 min	"The problem is distributing keys over N servers when N changes. With hash(key) mod N, adding or removing a server remaps almost every key, which causes a cache-miss storm. I want a scheme that moves only about k/N keys and stays balanced."
2. High-level design	8 min	"I hash both servers and keys onto a ring from 0 to 2^32-1. A key belongs to the first server clockwise. Adding a server takes keys only from its anticlockwise neighbour; removing a server gives its keys to the next one. Lookup is a binary search over the sorted ring."

Phase	Time	What to say
3. Deep dive	25 min	"The basic ring has uneven partitions and skewed key distribution, so I use virtual nodes: each server appears 100-200 times on the ring. That lowers variance, lets stronger machines take more vnodes, and spreads a failed node's load across many servers. For replication I take the next N distinct physical servers clockwise."
4. Wrap up	4 min	"This is the foundation for Dynamo, Cassandra, CDNs and cache clients. Remaining issues are hot keys, which hashing alone does not fix, and how clients learn about ring changes - via gossip or a config service."

11. 60-second recap / practice checklist

- Ring + clockwise lookup => only k/N keys move on membership change.
- Virtual nodes fix imbalance and spread failure load.
- Used in Dynamo, Cassandra, CDNs, caches, load balancers; know the add/remove walk-through.

12. References and further reading

Cross-checked against these sources (links verified when this PDF was built).

• Dynamo paper (SOSP 2007)
• GeeksforGeeks: Consistent hashing
• Karger et al.: Consistent hashing and random trees